Student Name: HARINI.S

Register Number: 192412098

Date:10/11/2026

CO1 ASSIGNMENT 1

GRAMa JYOTHI RURAL ELECTRICITY CO-OPERATIVE

Smart Electricity Billing System Using C

INTRODUCTION

Grama Jyothi is a village electricity co-operative that provides electricity to approximately 4,000 households every month.

The co-operative currently calculates electricity bills using previous and current meter readings. The existing spreadsheet system has several problems, such as incorrect calculations, floating-point precision errors, negative bills during meter rollover and difficulty updating electricity tariffs.

To solve these problems, a Smart Electricity Billing System is developed using the C programming language.

The system calculates electricity consumption, applies slab-based tariff rates, adds fixed charges and generates accurate electricity bills.

The original tariff structure is:

Electricity consumption	Energy charge

First 100 units	Free
101–200 units	₹2.25 per unit
Above 200 units	₹4.50 per unit
Fixed monthly charge	₹50.00

The system should also support meter rollover, accurate monetary calculations and large-scale billing for village, taluk and district levels.

Main objectives:

1. Calculate electricity consumption accurately.
2. Apply slab-based electricity tariff rates.
3. Avoid floating-point errors in financial calculations.
4. Handle 16-bit meter rollover.
5. Use appropriate C data types.
6. Implement reusable functions and macros.
7. Validate user inputs.
8. Support large numbers of electricity consumers.
9. Extend the system to smart-meter billing.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE DATA

Aim: Analyse the electricity billing problem and identify the required inputs, outputs, constants, variables and correctness rules.

1.1 Problem analysis

The electricity billing system receives the following information:

- Household number.
- Previous meter reading.

- Current meter reading.
- Household category.

The program calculates the electricity units consumed by subtracting the previous reading from the current reading.

After calculating consumption, the system applies the tariff rates and adds the fixed monthly charge.

Finally, it displays the complete electricity bill.

1.2 Identification of inputs and outputs

Data item	Description	Example
household_no	Unique household identification	1001
previous_reading	Previous meter reading	1200
current_reading	Current meter reading	1350
category	Household connection category	Domestic
units	Electricity consumed	150
energy_charge	Charge based on consumption	₹112.50
fixed_charge	Fixed monthly charge	₹50.00

total_bill	Final electricity bill	₹162.50

1.3 Identification of constants and variables

Constants: Values that do not change during program execution.

Constant	Value	Purpose
SLAB1_UNITS	100	First slab limit
SLAB2_UNITS	200	Second slab limit
RATE1_PAISE	0	First 100 units are free
RATE2_PAISE	225	₹2.25 per unit
RATE3_PAISE	450	₹4.50 per unit
FIXED_CHARGE_PAISE	5000	₹50 fixed charge
USHRT_MAX	65535	Maximum 16-bit unsigned reading

Variables: Values that change during program execution.

- Previous meter reading.
- Current meter reading.
- Electricity units consumed.
- Energy charge.

- Fixed charge.
- Total bill.

1.4 Correctness rules

The billing system must follow these rules:

1. Meter readings must be between 0 and 65535.
2. Electricity consumption must not be negative.
3. The first 100 units must be free.
4. Units from 101 to 200 must cost ₹2.25 per unit.
5. Units above 200 must cost ₹4.50 per unit.
6. The fixed charge must be ₹50.
7. Total bill must equal energy charge plus fixed charge.
8. The electricity bill must never be negative.
9. Monetary calculations must be accurate to the nearest paisa.

1.5 Assumptions

- Billing is performed monthly.
- The meter uses a 16-bit unsigned register.
- The maximum meter reading is 65535.
- The same tariff structure is applied to all households because separate category rates are not specified.
- The fixed monthly charge is ₹50.

A decrease in meter reading is treated as rollover only when the rollover rule or meter information confirms it. Otherwise, it is treated as invalid input.

1.6 Example calculation

Given:

Previous reading = 1200

Current reading = 1350

Step 1: Calculate units consumed

$$Units = Current - Previous$$

$$= 1350 - 1200 = 150$$

Step 2: Calculate energy charge

First 100 units are free.

Remaining units:

$$150 - 100 = 50$$

Energy charge:

$$50 \times 2.25 = ₹112.50$$

Step 3: Add fixed charge

$$Total = 112.50 + 50$$

Final Bill = ₹162.50

1.7 Alternative concept: Text readings versus integer readings

Meter readings can be stored as either integers or character strings.

Integer representation:

```
None
int previous = 1200;
int current = 1530;

int units = current - previous;

printf("Units consumed = %d", units);
```

Character string representation:

None

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char previous_text[] = "01200";
    char current_text[] = "01530";

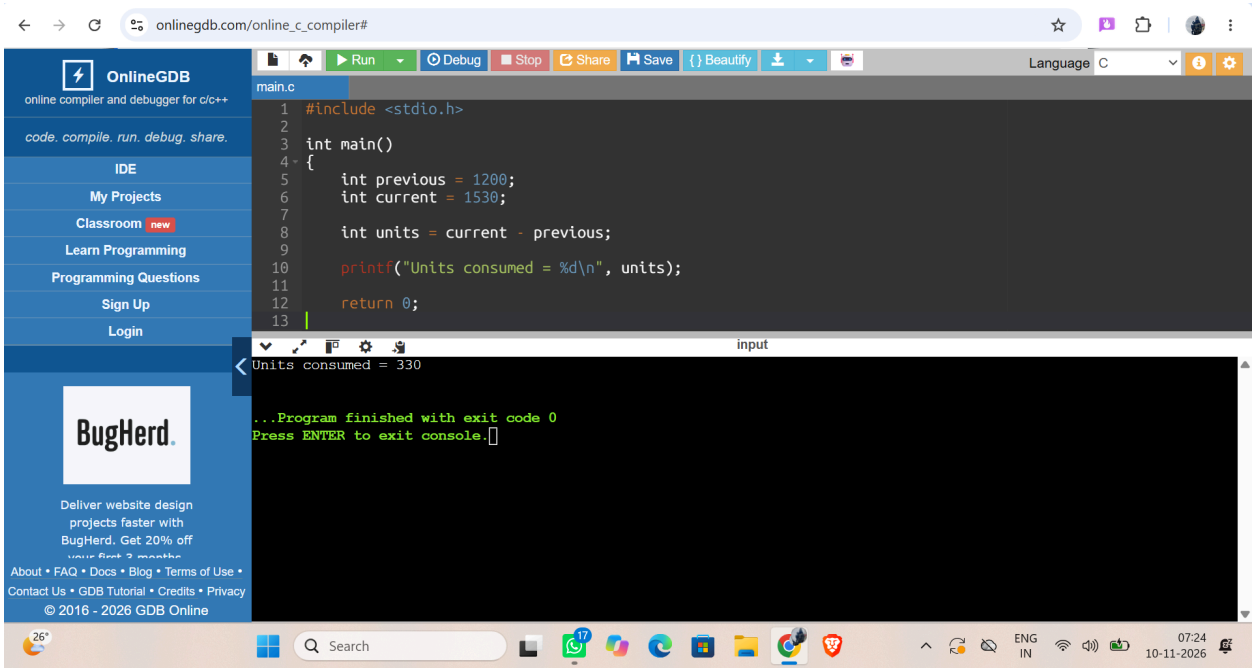
    int previous = atoi(previous_text);
    int current = atoi(current_text);

    int units = current - previous;

    printf("Units consumed = %d", units);

    return 0;
}
```

Output:



Comparison:

Feature	Integer	Character string
Calculation	Direct arithmetic	Requires conversion
Comparison	Numeric	Lexicographic unless converted
Time complexity	O(1) for fixed-width values	O(d) for d digits
Storage	Fixed number of bytes	O(d) characters
Suitability	Best for calculations	Useful for formatted input

Conclusion: Integer representation is more suitable for electricity billing because it provides faster and simpler calculations. Text input can be accepted and converted into integers after validation.

TASK 2 – CHOOSE THE DATA TYPES

Aim: Select appropriate C data types for storing meter readings, electricity units and monetary values.

2.1 Recommended data types

Data item	C data type	Reason
Meter reading	unsigned short	Represents a 16-bit meter

Household number	unsigned int	Stores positive identification numbers
Electricity units	unsigned int	Consumption is non-negative
Tariff rates	long / long long	Stores monetary values in paise
Individual bill	long	Stores bill amount in paise
District total	long long	Stores large accumulated totals
Text input	char array	Stores user-entered text

The exact sizes of data types depend on the compiler and operating system.

2.2 Complete rollover program without if-else

A 16-bit unsigned meter can store values from 0 to 65535.

When the meter reaches 65535, the next increment returns it to 0.

Program:

```
None
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous,
                         meter_t current)
{
```

```
    return (meter_t)(current - previous);
}

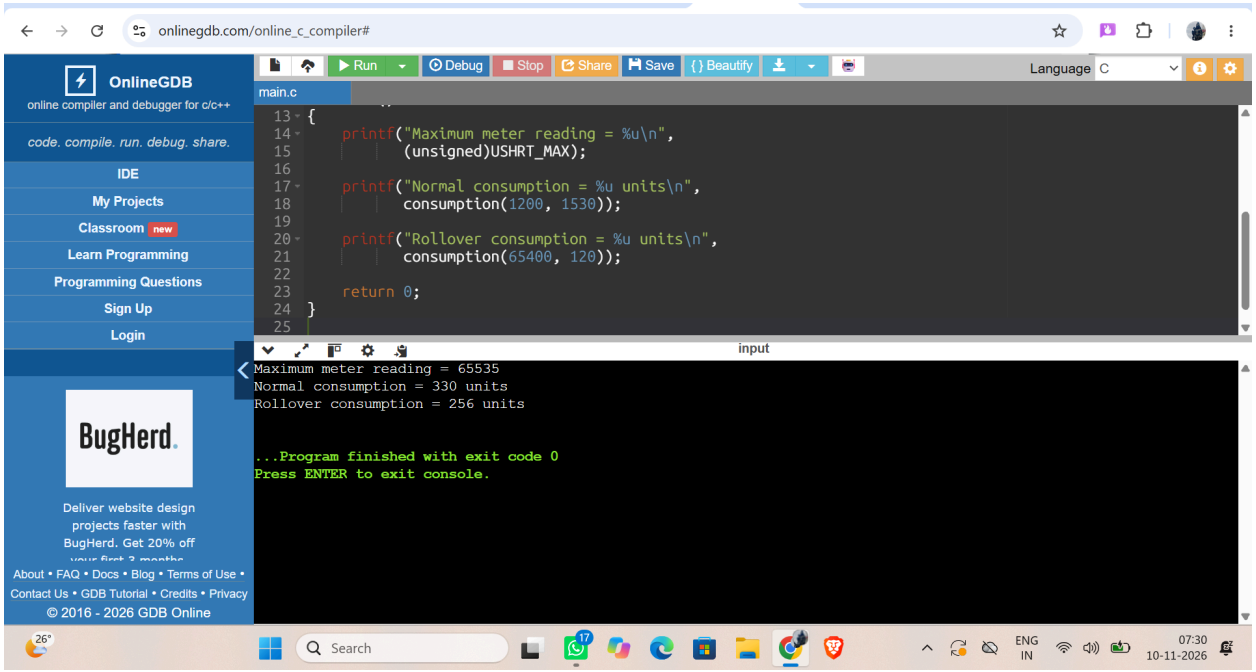
int main()
{
    printf("Maximum meter reading = %u\n",
           (unsigned)USHRT_MAX);

    printf("Normal consumption = %u units\n",
           consumption(1200, 1530));

    printf("Rollover consumption = %u units\n",
           consumption(65400, 120));

    return 0;
}
```

OUTPUT:



Explanation:

Normal consumption:

$$1530 - 1200 = 330$$

Rollover consumption:

$$(65535 - 65400) + 1 + 120$$

$$= 135 + 1 + 120 = 256$$

This uses unsigned modular arithmetic and assumes `USHRT_MAX` is 65535.

2.3 Inspect the size and range of data types

Program:

```
None
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main()
{
    printf("Size of char = %zu\n", sizeof(char));

    printf("Size of short = %zu\n",
           sizeof(short));

    printf("Maximum unsigned short = %u\n",
           (unsigned)USHRT_MAX);

    printf("Size of int = %zu\n", sizeof(int));

    printf("INT_MAX = %d\n", INT_MAX);
```

```
    printf("Size of long = %zu\n", sizeof(long));

    printf("LONG_MAX = %ld\n", LONG_MAX);

    printf("Size of long long = %zu\n",
           sizeof(long long));

    printf("LLONG_MAX = %lld\n", LLONG_MAX);

    printf("Size of float = %zu\n", sizeof(float));

    printf("FLT_DIG = %d\n", FLT_DIG);

    printf("Size of double = %zu\n",
           sizeof(double));

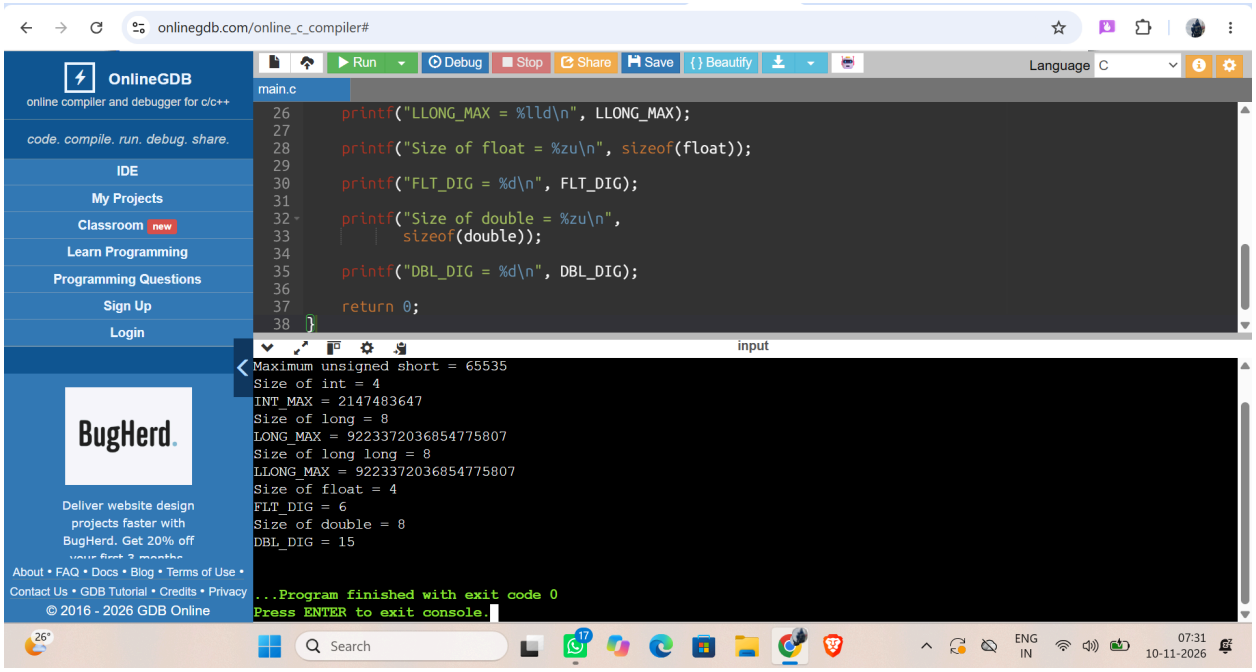
    printf("DBL_DIG = %d\n", DBL_DIG);

    return 0;
}
```

Explanation:

- `sizeof()` determines the memory size of a data type.
- `limits.h` provides integer limits.
- `float.h` provides floating-point characteristics.
- `USHRT_MAX` gives the maximum unsigned short value.
- `LLONG_MAX` gives the maximum long long value.

OUTPUT:



2.4 Why should float not be used for money?

Floating-point numbers cannot represent many decimal fractions exactly in binary.

For example, repeated addition of ₹0.01 may produce a slightly inaccurate result.

Instead, money should be stored in integer paise.

Example:

₹162.50 = 16250 paise.

Program:

```
None
#include <stdio.h>

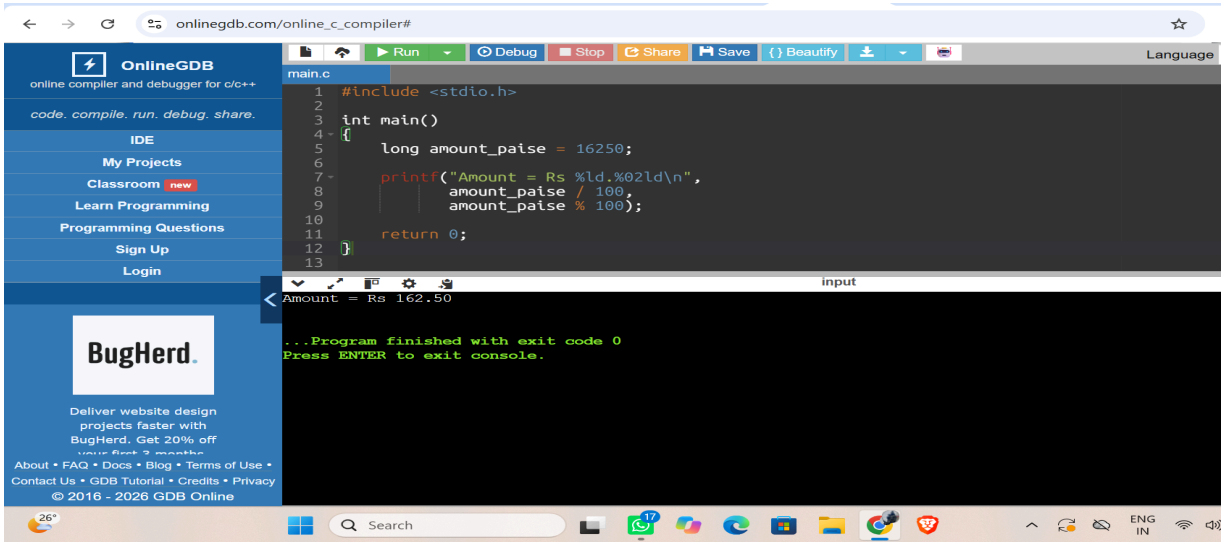
int main()
{
    long amount_paise = 16250;

    printf("Amount = Rs %ld.%02ld\n",
```

```
            amount_paise / 100,
            amount_paise % 100);

    return 0;
}
```

OUTPUT:



2.5 Float versus integer precision experiment

Program:

```
None
#include <stdio.h>

int main()
{
    float total_float = 0.0f;
    long total_paise = 0;

    for(int i = 0; i < 4000; i++)
```

```
    {
        total_float += 0.01f;
        total_paise += 1;
    }

    printf("Float total = %.8f\n",
           total_float);

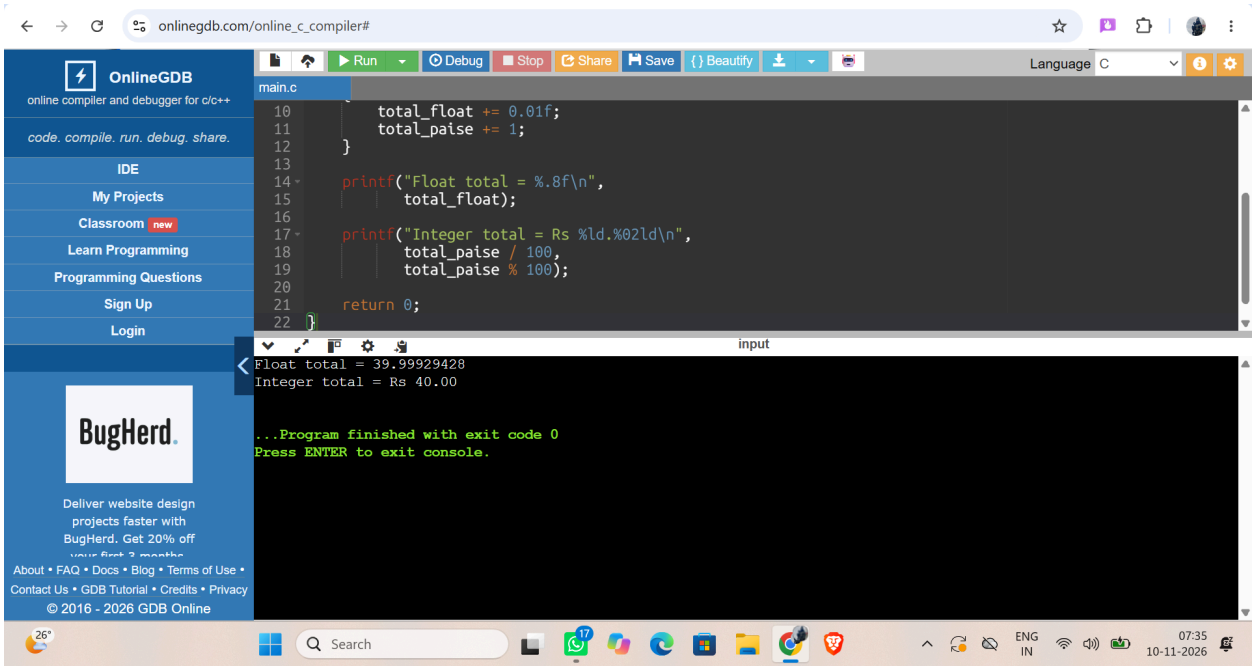
    printf("Integer total = Rs %ld.%02ld\n",
           total_paise / 100,
           total_paise % 100);

    return 0;
}
```

Expected mathematical result:

```
None
Integer total = Rs 40.00
```

OUTPUT:



Conclusion:

Integer paise is the best approach for financial calculations. `long long` is recommended for large district-level totals.

TASK 3 – DEFINE THE CONSTANTS

Aim: Define electricity tariff values in one place using preprocessor macros.

3.1 Why are constants required?

Constants prevent repeated tariff values from being written throughout the program.

If the electricity board changes its tariff, the programmer can modify the value in one place instead of changing several calculations.

3.2 Complete C program

```
None
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
```

```
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if(units > SLAB2_UNITS)
    {
        paise += (long)(SLAB2_UNITS - SLAB1_UNITS)
                 * RATE2_PAISE;

        paise += (long)(units - SLAB2_UNITS)
                 * RATE3_PAISE;
    }
    else if(units > SLAB1_UNITS)
    {
        paise += (long)(units - SLAB1_UNITS)
                 * RATE2_PAISE;
    }

    return paise;
}

int main()
{
    int tests[] = {0, 100, 150, 200, 350};

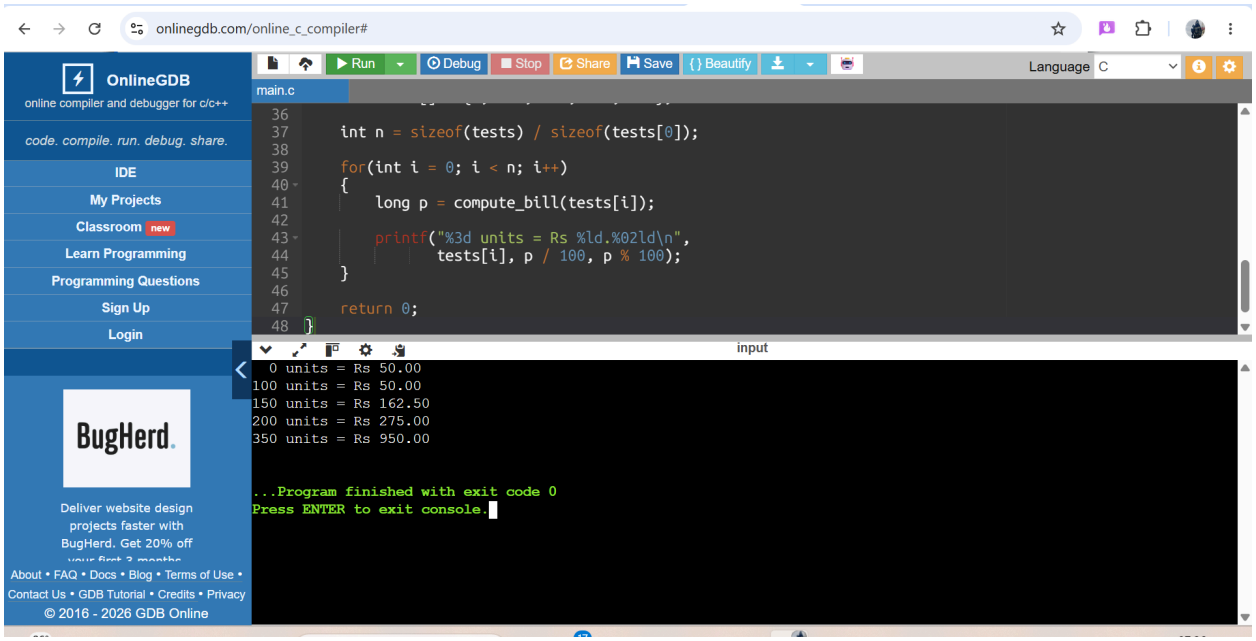
    int n = sizeof(tests) / sizeof(tests[0]);
```

```
    for(int i = 0; i < n; i++)
    {
        long p = compute_bill(tests[i]);

        printf("%3d units = Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }

    return 0;
}
```

OUTPUT:



3.4 Verify tariff revision

Suppose the electricity board changes the second slab rate from ₹2.25 to ₹2.30.

Change:

```
None
#define RATE2_PAISE 230
```

New calculations:

For 150 units:

$$50 \times 2.30 + 50 = ₹165.00$$

For 200 units:

$$100 \times 2.30 + 50 = ₹280.00$$

For 350 units:

$$(100 \times 2.30) + (150 \times 4.50) + 50 = ₹955.00$$

Note: The correct revised bill for 150 units is ₹165.00.

3.5 Comparison of constants

Method	Advantage	Disadvantage
`#define`	Simple macro substitution	No normal type checking
`const`	Type checked by compiler	Follows variable scope rules
`enum`	Useful for integer constants	Only integral values

Conclusion: Using named constants improves program readability, reduces errors and makes tariff modifications easier.

TASK 4 – USE MACROS SAFELY

Aim: Understand preprocessor macros, write safe macros and compare macros with inline functions.

4.1 Safe macro definitions

Macros are defined using the `#define` preprocessor directive.

Program:

```
None
#include <stdio.h>

#define RUPEES_TO_PAISE(x) ((long)(x) * 100L)

#define SQUARE(x) ((x) * (x))

#define MAX(a,b) ((a) > (b) ? (a) : (b))

int main()
{
    int a = 4, b = 7;

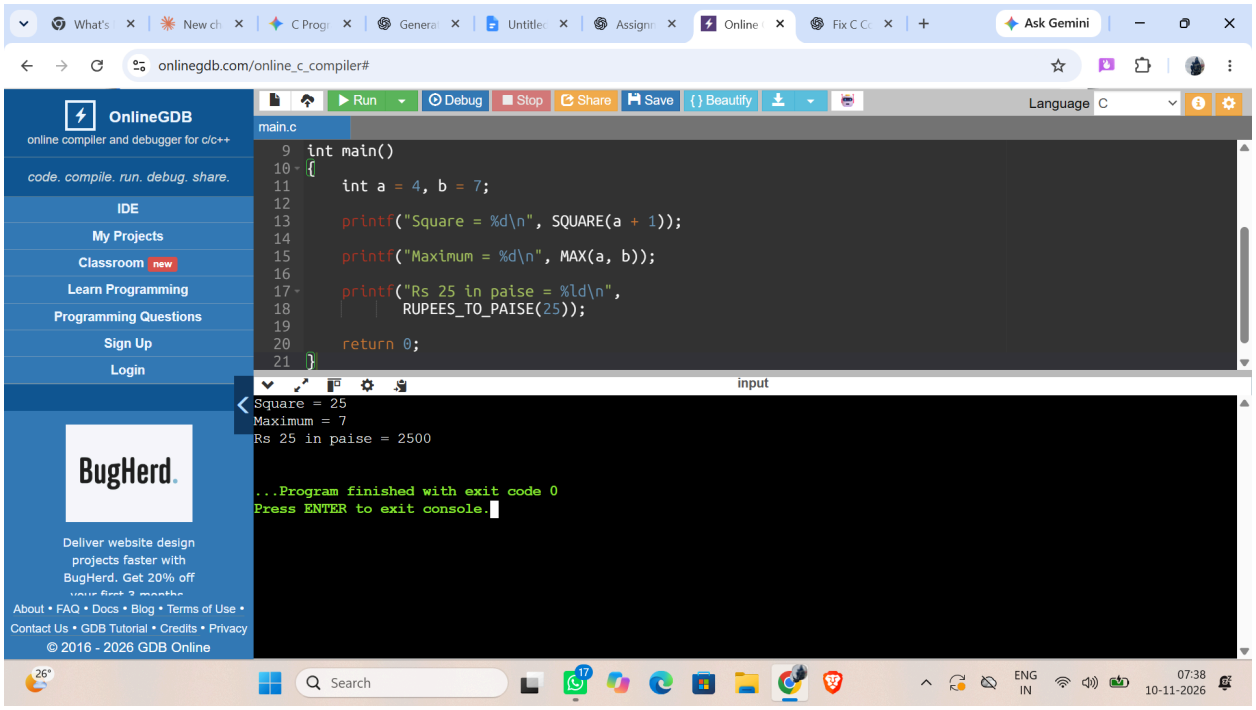
    printf("Square = %d\n", SQUARE(a + 1));

    printf("Maximum = %d\n", MAX(a, b));

    printf("Rs 25 in paise = %ld\n",
           RUPEES_TO_PAISE(25));

    return 0;
}
```

OUTPUT:



4.3 Macro pitfalls

1. Incorrect parentheses

Unsafe macro:

```
None
#define SQUARE(x) x*x
```

When called as:

```
None
SQUARE(a+1)
```

It expands to:

```
None
a+1*a+1
```

This produces an incorrect result because multiplication has higher precedence than addition.

Correct macro:

```
None
#define SQUARE(x) ((x)*(x))
```

2. Multiple evaluation

Consider:

```
None
#define MAX(a,b) ((a)>(b)?(a):(b))
```

Calling `MAX(a++, b)` may increment `a` more than once.

Therefore, avoid arguments that contain side effects.

3. Lack of type checking

Macros perform textual replacement and do not provide normal function type checking.

4.4 Preprocessor expansion

Command:

```
None
gcc -E macro_demo.c -o macro_demo.i
```

Explanation:

- `gcc -E` performs preprocessing.
- Macro names are replaced with their defined expressions.
- The expanded code is saved in `macro_demo.i`.

4.5 Inline-function alternative

```
None
static inline int square_int(int x)
{
    return x * x;
```

```
}

static inline int max_int(int a, int b)
{
    return a > b ? a : b;
}
```

Comparison:

Feature	Macro	Inline function
Type checking	No normal checking	Yes
Argument evaluation	May happen multiple times	Once per parameter
Execution	Text substitution	Compiler may inline
Debugging	Difficult	Easier
Safety	Less safe	More safe

Conclusion: Macros are useful for simple constants and expressions. Inline functions are preferable for complex calculations.

TASK 5 – ORGANISE THE PROGRAM WITH HEADER FILES AND CONDITIONAL COMPILATION

Aim: Divide the billing system into separate source files using header files and conditional compilation.

5.1 Program file structure

Create a folder named `billing_project`.

Inside the folder, create three files:

```
None
billing_project/
│
├── main.c
├── billing.c
└── billing.h
```

5.2 billing.h

This header file contains tariff constants and the function declaration.

```
None
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units);

#endif
```

Explanation:

- `#ifndef` checks whether the header has already been included.
- `#define BILLING_H` defines the header guard.
- `#endif` closes the conditional directive.
- `compute_bill()` is declared for use in other source files.

5.3 billing.c

This file contains the actual billing calculation.

```
None
#include <stdio.h>
#include "billing.h"

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

#ifdef DEBUG
    printf("[DEBUG] Units = %d\n", units);
#endif

    if(units > SLAB2_UNITS)
    {
        paise += 100L * RATE2_PAISE;

        paise += (long)(units - SLAB2_UNITS)
                 * RATE3_PAISE;
    }
    else if(units > SLAB1_UNITS)
    {
        paise += (long)(units - SLAB1_UNITS)
                 * RATE2_PAISE;
    }

#ifdef DEBUG
```

```
    printf("[DEBUG] Bill in paise = %ld\n", paise);
#endif

    return paise;
}
```

5.4 main.c

This file contains the main function.

```
None
#include <stdio.h>
#include "billing.h"

int main()
{
    int units = 150;

    long p = compute_bill(units);

    printf("Bill = Rs %ld.%02ld\n",
           p / 100, p % 100);

    return 0;
}
```

5.5 Compile with and without DEBUG

Normal compilation:

```
None
gcc -Wall -Wextra main.c billing.c -o billing
```

Run:

```
None
./billing
```

Output:

```
None
Bill = Rs 162.50
```

Compilation with debugging:

```
None
gcc -Wall -Wextra -DDEBUG main.c billing.c -o billing_debug
```

Run:

```
None
./billing_debug
```

Output:

```
None
[DEBUG] Units = 150
[DEBUG] Bill in paise = 16250
Bill = Rs 162.50
```

5.6 Advantages of header files

1. Code can be divided into separate modules.
2. Functions can be reused.
3. Maintenance becomes easier.
4. Header guards prevent repeated declarations.
5. Debugging becomes simpler.
6. Conditional compilation allows optional debugging statements.

Conclusion: Header files and separate source files improve modularity, maintainability and reusability.

TASK 6 – READ AND PRINT DATA CORRECTLY

Aim: Use correct format specifiers and validate user input using `scanf()` and `printf()`.

6.1 Common C format specifiers

Data type	Format specifier	Example
int	`%d`	`scanf("%d",&units)`
unsigned int	`%u`	`scanf("%u",&n)`
long	`%ld`	`printf("%ld",amount)`
long long	`%lld`	`printf("%lld",total)`
float	`%f`	`scanf("%f",&x)`
double	`%lf` in scanf, `%f` in printf	`scanf("%lf",&x)`
char	`%c`	`scanf("%c",&c)`

String	%s	scanf("%s",name)

6.2 C program for input validation

```
None
#include <stdio.h>

int main()
{
    int household, previous, current, units;

    printf("Enter household number: ");

    if(scanf("%d", &household) != 1)
    {
        printf("Invalid household number\n");
        return 1;
    }

    printf("Enter previous reading: ");

    if(scanf("%d", &previous) != 1 ||
       previous < 0)
    {
        printf("Invalid previous reading\n");
        return 1;
    }

    printf("Enter current reading: ");

    if(scanf("%d", &current) != 1 ||
       current < 0)
```

```
    {
        printf("Invalid current reading\n");
        return 1;
    }

    if(current < previous)
    {
        printf("Reading decreased: Check rollover\n");
        return 1;
    }

    units = current - previous;

    printf("Household number = %d\n", household);

    printf("Units consumed = %d\n", units);

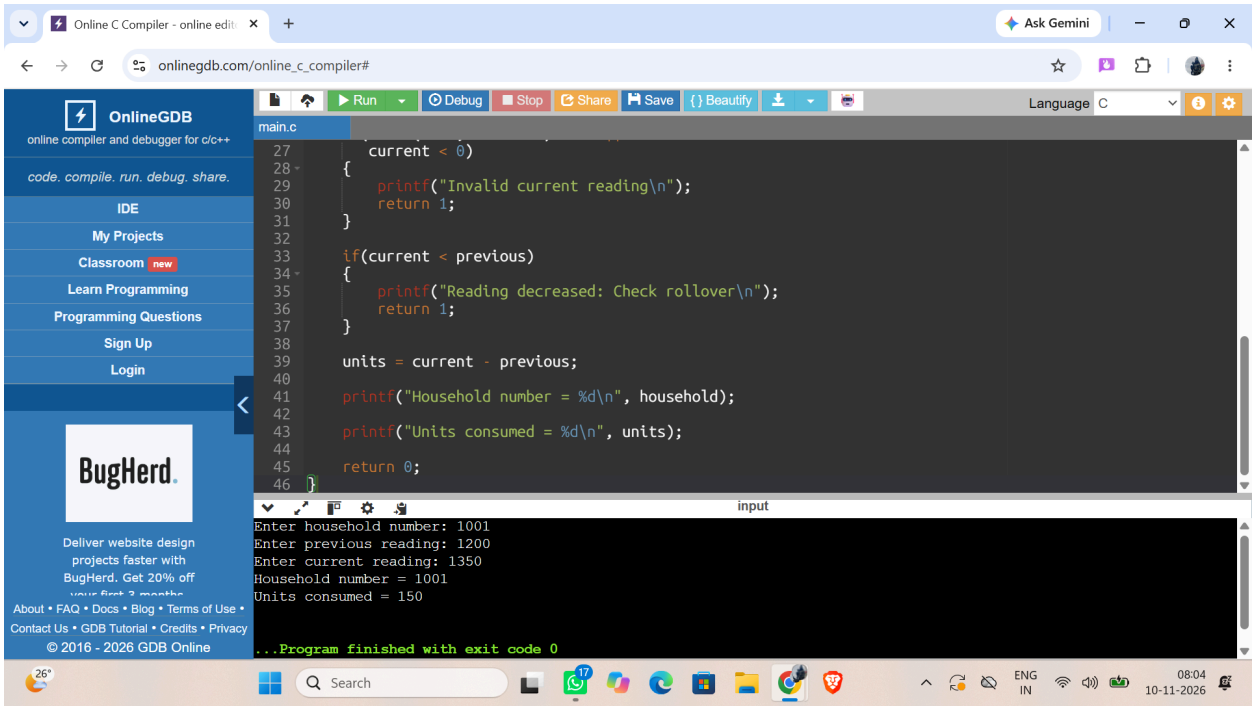
    return 0;
}
```

6.3 Sample input

None

```
Enter household number: 1001
Enter previous reading: 1200
Enter current reading: 1350
```

OUTPUT:



6.4 Invalid input example

Input:

```
None
Enter household number: abc
```

Output:

```
None
Invalid household number
```

6.5 Importance of input validation

Input validation prevents incorrect calculations.

For example:

- Negative readings must be rejected.
- Alphabetic characters must not be accepted as meter readings.
- Readings greater than 65535 must be rejected for the specified meter.
- Incorrect format specifiers must be avoided.

For production software, `fgets()` with `strtol()` is preferable because it provides better control over invalid text, trailing characters and numeric overflow.

Conclusion: Correct format specifiers and input validation ensure that the billing system processes only valid data.

TASK 7 – CONTROL TYPE CONVERSION AND PRECISION

Aim: Understand implicit conversion, explicit conversion, integer division and floating-point precision.

7.1 C program demonstrating type conversion

```
None
#include <stdio.h>

int main()
{
    printf("5 / 2 = %d\n", 5 / 2);

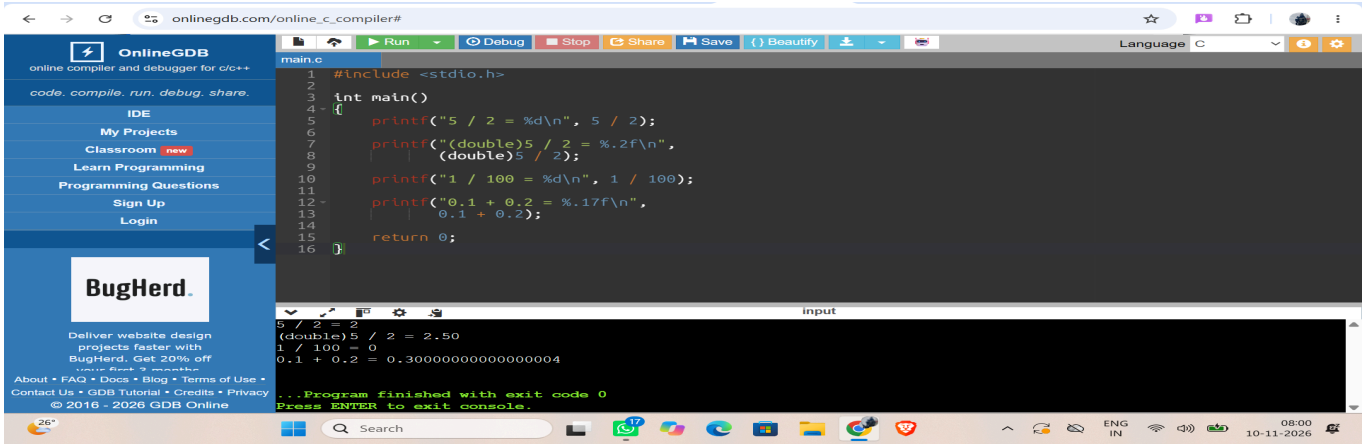
    printf("(double)5 / 2 = %.2f\n",
           (double)5 / 2);

    printf("1 / 100 = %d\n", 1 / 100);

    printf("0.1 + 0.2 = %.17f\n",
           0.1 + 0.2);

    return 0;
}
```

OUTPUT:



7.3 Explanation

1. Integer division

```
None
5 / 2
```

Both operands are integers, so the fractional part is removed.

Result:

```
None
2
```

2. Explicit type conversion

```
None
(double)5 / 2
```

The first operand is converted into double.

Result:

```
None
2.50
```

3. Integer division with a small fraction

```
None
1 / 100
```

Both operands are integers, so the result is zero.

4. Floating-point precision

```
None
0.1 + 0.2
```

The binary representation of these decimal values is not exact. Therefore, the result may be displayed as:

```
None
0.30000000000000004
```

7.4 Correct monetary calculation

The electricity billing system should use integer paise.

For example:

```
None
long bill = 16250;

printf("Bill = Rs %ld.%02ld\n",
       bill / 100,
       bill % 100);
```

Output:

```
None
Bill = Rs 162.50
```

7.5 Importance of type conversion

- Integer division removes decimal portions.
- Explicit casting can produce floating-point results.
- Floating-point calculations may introduce small precision errors.
- Converting large values into smaller integer types may cause data loss.
- Integer paise avoids binary floating-point errors in ordinary monetary calculations.

Conclusion: The billing system should use integer arithmetic for money and apply a clearly defined rounding policy whenever fractional paise arise.

TASK 8 – TRACE THE COMPILATION PROCESS

Aim: Understand the different stages of compiling a C program using GCC.

8.1 Four stages of compilation

The GCC compiler performs four major stages:

1. Preprocessing.
2. Compilation.
3. Assembly.
4. Linking.

8.2 GCC commands

Stage 1 – Preprocessing

```
None
gcc -E main.c -o main.i
```

This expands macros and processes header files.

Stage 2 – Compilation

```
None
gcc -S main.c -o main.s
```

This converts the C source into assembly language.

Stage 3 – Assembly

```
None
gcc -c main.c -o main.o
```

This creates an object file.

Stage 4 – Linking

```
None
gcc main.o billing.o -o billing
```

This combines object files and required libraries to generate the executable.

For this example, first create `billing.o` using:

```
None
gcc -c billing.c -o billing.o
```

8.3 Types of errors

Error type	Example	Description
Compile-time error	Missing semicolon	Compiler cannot translate the source
Link-time error	Undefined function	Linker cannot find the required function definition
Run-time error	Invalid file operation	Error occurs during execution

8.4 Advantages of separate compilation

1. Large programs can be divided into smaller files.

2. Only modified files need recompilation.
3. Debugging becomes easier.
4. Code maintenance becomes simpler.
5. Multiple programmers can work on different modules.

Conclusion: Understanding GCC compilation stages helps programmers identify errors and develop modular C applications.

TASK 9 – IMPLEMENT AND TEST THE BILLING PROGRAM

Aim: Develop a complete electricity billing system and test normal readings, boundary values, rollover and invalid input.

9.1 Complete C billing program

The following program includes:

- Household number.
- Previous meter reading.
- Current meter reading.
- Meter rollover handling.
- Input validation.
- Slab-based billing.
- Fixed monthly charge.
- Final bill calculation.

```
None
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
```

```
#define FIXED_CHARGE_PAISE 5000L

#define ROLLOVER_PREVIOUS_MIN 60000U
#define ROLLOVER_CURRENT_MAX 1000U

unsigned int get_units(meter_t previous,
                       meter_t current)
{
    if(current >= previous)
    {
        return (unsigned int)(current - previous);
    }

    if((unsigned int)previous >= ROLLOVER_PREVIOUS_MIN &&
       (unsigned int)current <= ROLLOVER_CURRENT_MAX)
    {
        return (unsigned int)
               (meter_t)(current - previous);
    }

    return UINT_MAX;
}

long compute_bill(unsigned int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if(units > SLAB2_UNITS)
    {
        paise += 100L * RATE2_PAISE;

        paise += (long)(units - SLAB2_UNITS)
                 * RATE3_PAISE;
```

```
    }
    else if(units > SLAB1_UNITS)
    {
        paise += (long)(units - SLAB1_UNITS)
                 * RATE2_PAISE;
    }

    return paise;
}

int main()
{
    unsigned int household;
    unsigned int p, c, units;

    printf("Enter household number: ");

    if(scanf("%u", &household) != 1)
    {
        printf("Invalid household number\n");
        return 1;
    }

    printf("Enter previous reading (0-65535): ");

    if(scanf("%u", &p) != 1 || p > USHRT_MAX)
    {
        printf("Invalid previous reading\n");
        return 1;
    }

    printf("Enter current reading (0-65535): ");
```

```
if(scanf("%u", &c) != 1 || c > USHRT_MAX)
{
    printf("Invalid current reading\n");
    return 1;
}

units = get_units((meter_t)p, (meter_t)c);

if(units == UINT_MAX)
{
    printf("Invalid reading decrease\n");
    return 1;
}

long bill = compute_bill(units);

long energy = bill - FIXED_CHARGE_PAISE;

printf("\n----- ELECTRICITY BILL -----\n");

printf("Household number: %u\n", household);

printf("Units consumed: %u\n", units);

printf("Energy charge: Rs %ld.%02ld\n",
       energy / 100, energy % 100);

printf("Fixed charge: Rs %ld.%02ld\n",
       FIXED_CHARGE_PAISE / 100,
       FIXED_CHARGE_PAISE % 100);

printf("Total bill: Rs %ld.%02ld\n",
       bill / 100, bill % 100);
```

```
    return 0;
}
```

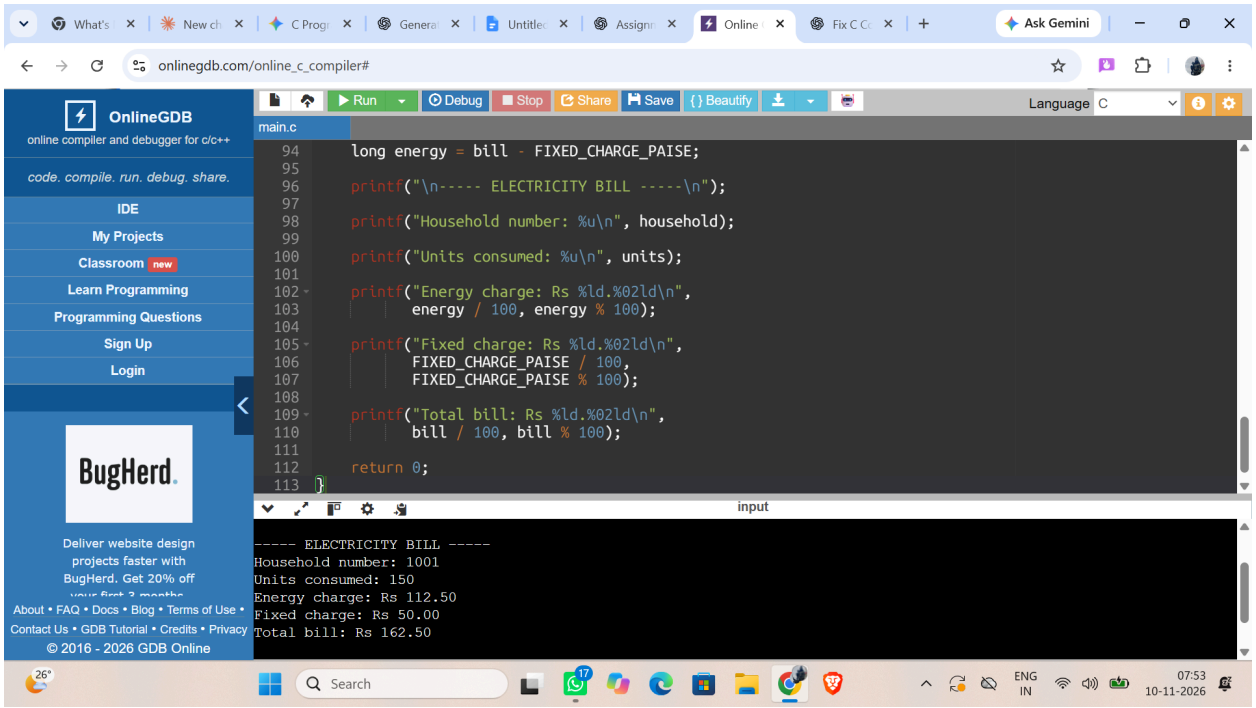
The rollover thresholds are application assumptions used to distinguish likely rollover from an invalid decrease. The assignment specifically requires a test from 65535 to 0.

9.2 Sample output – Normal reading

Input:

```
None
Enter household number: 1001
Enter previous reading (0-65535): 1200
Enter current reading (0-65535): 1350
```

OUTPUT:



9.3 Test cases

Test case	Previous	Current	Expected result
Normal reading	1200	1350	150 units
First boundary	1000	1100	100 units
Second boundary	1000	1200	200 units
Zero consumption	1000	1000	0 units
Rollover	65535	0	1 unit
Invalid decrease	3000	2500	Rejected
Negative reading	-5	—	Rejected
Alphabetic input	abc	—	Rejected

9.4 Five-household hand calculation

Assume five households consume different amounts of electricity.

Household	Units	Energy charge	Fixed charge	Total bill
A	0	₹0.00	₹50.00	₹50.00

B	100	₹0.00	₹50.00	₹50.00
C	150	₹112.50	₹50.00	₹162.50
D	200	₹225.00	₹50.00	₹275.00
E	350	₹900.00	₹50.00	₹950.00
Total		₹1,237.50	₹250.00	₹1,487.50

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION

Aim: Compare floating-point and integer calculations and evaluate the system's performance at village, taluk and district levels.

10.1 Scale analysis

The electricity billing system must support three different levels:

1. Village – 4,000 households.
2. Taluk – 60,000 households.
3. District – 800,000 households.

For this example, assume each household receives a bill of ₹950.

Calculation:

For one household:

$$Bill = ₹950$$

For 4,000 households:

$$4000 \times 950 = ₹38,00,000$$

For 60,000 households:

$$60000 \times 950 = ₹5,70,00,000$$

For 800,000 households:

$$800000 \times 950 = ₹76,00,00,000$$

10.2 Comparison table

Level	Households	Total amount
Village	4,000	₹38,00,000
Taluk	60,000	₹5,70,00,000
District	800,000	₹76,00,00,000

These are sample estimates, not actual billing records.

10.3 C program for large-scale total calculation

```
None
#include <stdio.h>

int main()
{
    const int counts[] = {
```

```
        4000, 60000, 800000
    };

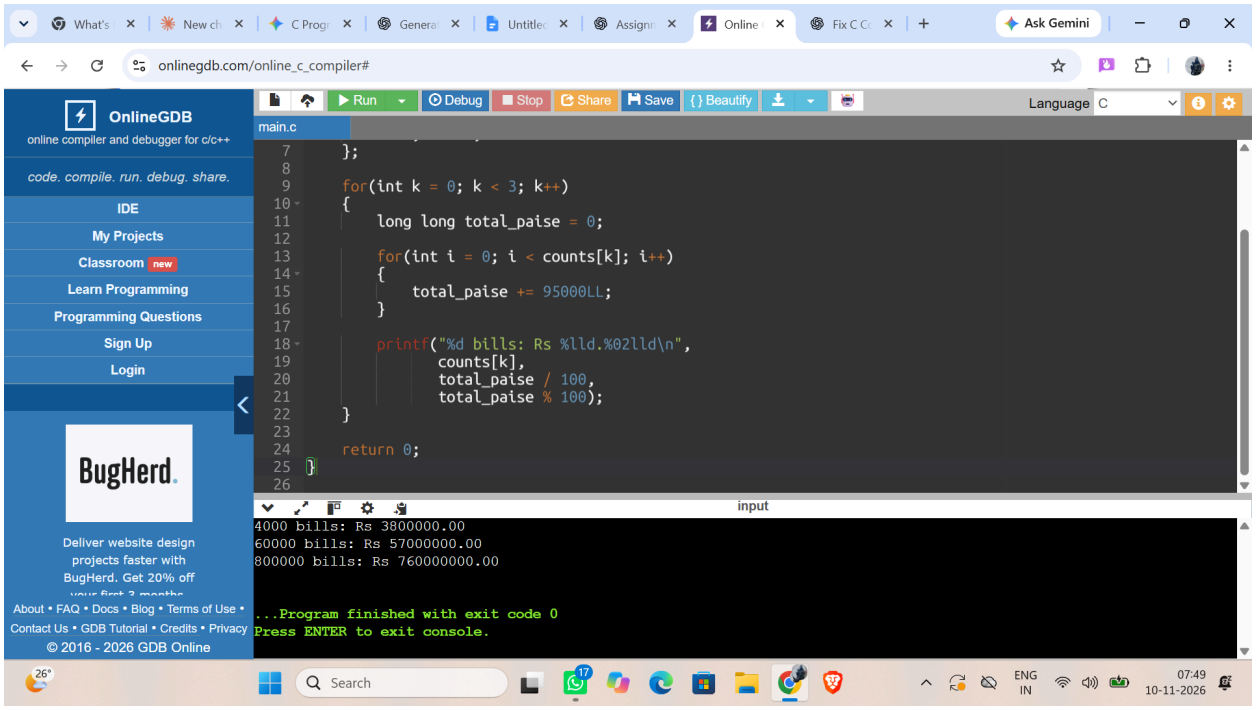
    for(int k = 0; k < 3; k++)
    {
        long long total_paise = 0;

        for(int i = 0; i < counts[k]; i++)
        {
            total_paise += 95000LL;
        }

        printf("%d bills: Rs %lld.%02lld\n",
               counts[k],
               total_paise / 100,
               total_paise % 100);
    }

    return 0;
}
```

OUTPUT:



10.5 Why is long long required?

A normal 32-bit integer cannot store the district-level total in paise.

For 800,000 households:

$$800000 \times 95000 = 76,000,000,000 paise$$

This value exceeds the maximum capacity of a signed 32-bit integer.

Therefore, `long long` is recommended.

Data type	Suitability
int	Insufficient for the district total in paise
long	Depends on platform size

long long	Suitable for the calculated district total
float	Not recommended for exact financial totals
double	Wider precision but still binary floating-point

Conclusion:

The system should use `long long` for large accumulated monetary totals to prevent integer overflow and preserve exact paise values.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

Aim: Recommend the most suitable data types, constants, macros and program structure for the electricity billing system.

11.1 Final design recommendation

After analysing the billing requirements and performing the calculations, the following design is recommended.

Design component	Recommended solution	Justification
Meter reading	unsigned short	Represents the specified 16-bit register
Electricity units	unsigned int	Consumption is non-negative
Individual bill	long integer paise	Accurate monetary calculation

District total	long long	Handles large totals
Tariff constants	`#define`	Centralised tariff management
Debugging	`#ifdef DEBUG`	Optional debugging output
Input validation	`fgets()` and `strtol()`	Better input checking
Program structure	Separate `.c` and `.h` files	Easy maintenance
Meter rollover	Validated modular arithmetic	Prevents negative consumption

11.2 Limitations of the existing design

Although the billing system is effective, some limitations remain.

1. The system assumes monthly billing.
2. It assumes whole-unit meter readings.
3. A decrease in readings may be caused by rollover or incorrect data entry.
4. Hard-coded tariff values require recompilation after a tariff revision.
5. The 16-bit meter format must be confirmed against the actual meter specification.
6. The system needs additional features for missing readings and duplicate records.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN – SMART METER CHALLENGE

Aim: Modify the existing electricity billing system to support smart meters, 15-minute readings, peak-hour tariffs and configurable rates.

12.1 Changed requirements

The electricity co-operative is planning to introduce smart meters.

The new requirements are:

1. Smart meters record electricity readings every 15 minutes.
2. The system must process 35,040 readings per household per year.
3. Electricity consumed between 6 PM and 10 PM costs 20% more.
4. Tariff rates must be changeable without recompiling the program.
5. The system must calculate normal and peak-hour consumption separately.

The number of annual readings is:

$$24 \times 4 \times 365 = 35040$$

Therefore, each household produces 35,040 readings per year.

12.2 Should the existing design be retained, modified or redesigned?

Recommendation: MODIFY the existing design.

A complete redesign is not necessary because the original billing system already supports important features such as integer arithmetic, input validation, tariff calculation and modular programming.

However, the existing system must be modified to support timestamped readings and time-based tariffs.

The following features should be retained:

- Integer-based monetary calculations.
- Input validation.
- Modular C programming.
- Large integer totals.
- Centralised tariff management.

The following features must be modified:

- Monthly meter readings should be replaced by 15-minute smart-meter readings.
- Peak-hour and normal-hour energy must be calculated separately.
- The 20% peak-hour premium must be applied.
- Tariff values must be loaded from an external configuration file.

12.3 Data-type selection for smart meters

Data item	Recommended type	Reason
Meter reading in Wh	unsigned long long	Stores large cumulative readings
Energy difference	unsigned long long	Stores non-negative interval consumption
Yearly reading count	unsigned int	Stores 35,040 readings
Timestamp	time_t or structured date/time	Identifies peak and normal periods
Tariff rate	Integer paise or scaled integer	Avoids floating-point money calculations
Total amount	long long	Stores large financial totals

The original 16-bit meter type should only be retained if the actual smart meter uses that register format and its rollover behaviour is properly handled.

12.4 Peak-hour and normal-hour calculation

The electricity board specifies the following time-based tariff:

Time period	Tariff
Normal hours	Regular tariff
6 PM to 10 PM	20% additional charge

Peak-hour multiplier:

\[100\%+20\%=120\% \]

$$Peak\ Rate = Normal\ Rate \times 1.20$$

Assume the normal tariff is ₹2.25 per unit.

Then:

$$Peak\ Rate = 2.25 \times 1.20$$

\[=\boxed{₹2.70} \]

Example calculation:

Suppose the smart meter records 250 Wh of electricity consumption.

Conversion into kWh:

$$250/1000 = 0.25kWh$$

Normal-hour charge:

$$0.25 \times 2.25 = ₹0.5625$$

Peak-hour charge:

$$0.25 \times 2.70 = ₹0.675$$

These amounts contain fractions of a paisa. Therefore, the system must accumulate energy and apply a clearly documented rounding rule, preferably rounding once at the final billing stage.

12.5 Configuration file for smart-meter billing

Create a file named:

`tariff_config.txt`

Content:

```
None
SLAB1_UNITS=100
SLAB2_UNITS=200

RATE1_PAISE=0
RATE2_PAISE=225
RATE3_PAISE=450

FIXED_CHARGE_PAISE=5000

PEAK_MULTIPLIER_NUM=120
PEAK_MULTIPLIER_DEN=100

PEAK_START_HOUR=18
PEAK_END_HOUR=22
```

Explanation:

- `SLAB1_UNITS` specifies the first slab.
- `SLAB2_UNITS` specifies the second slab.
- `RATE2_PAISE` stores the second-slab tariff.
- `RATE3_PAISE` stores the third-slab tariff.
- `FIXED_CHARGE_PAISE` stores the fixed charge.
- `PEAK_MULTIPLIER_NUM` represents 120%.
- `PEAK_MULTIPLIER_DEN` represents 100%.
- `PEAK_START_HOUR` specifies the beginning of peak hours.

- `PEAK_END_HOUR` specifies the end of peak hours.

The program should read and validate these values during startup.

12.6 Pseudocode for smart-meter processing

```
None
START

READ tariff_config.txt

VALIDATE tariff values

FOR each household

    normal_energy = 0
    peak_energy = 0

    FOR each 15-minute reading

        READ timestamp
        READ meter reading

        VALIDATE reading and timestamp

        CALCULATE energy difference

        IF time >= 18:00 AND time < 22:00

            peak_energy += energy difference

        ELSE

            normal_energy += energy difference

        END IF
```

```
    END FOR

    CALCULATE normal charge

    CALCULATE peak charge

    APPLY 20% peak premium

    APPLY documented rounding policy

    ADD fixed charge

    DISPLAY final electricity bill

END FOR

STOP
```

12.7 Advantages of the modified smart-meter system

1. More accurate electricity consumption monitoring.
2. Supports automatic 15-minute readings.
3. Provides separate peak-hour and normal-hour calculations.
4. Encourages consumers to reduce electricity consumption during peak hours.
5. Tariff rates can be changed without recompiling.
6. Supports large numbers of consumers.
7. Improves billing transparency.
8. Provides a foundation for future smart-grid applications.

12.8 Limitations and assumptions

The assignment specifies the peak period and premium but does not completely define how the original monthly slabs should be combined with the new time-of-day tariff.

Therefore, the electricity board must decide whether slabs apply to total consumption or separately to peak and normal consumption.

The final system must also define the rounding policy and handling of missing or duplicate readings.

CONCLUSION

The Grama Jyothi Rural Electricity Co-operative Smart Electricity Billing System demonstrates how C programming can be used to develop an accurate and reliable electricity billing application.

Through the twelve tasks, the assignment identifies the billing requirements, selects appropriate data types, defines tariff constants, implements safe macros, uses header files, validates input and develops a complete billing program.

The system also demonstrates the importance of integer-based monetary calculations to avoid floating-point precision errors.

The final design uses `unsigned short` for the specified 16-bit meter register, integer paise for individual bills and `long long` for large accumulated totals.

The system can be extended to support 15-minute smart-meter readings and time-of-day tariffs.

Therefore, the proposed solution provides a reliable and scalable foundation for village, taluk and district-level electricity billing.